

SESSION LIVE • 1 HEURE • ZÉRO CLOUD

LLM & développement agentique

Comprendre les modèles de bout en bout — et les faire tourner chez soi.

m-max • octobre 2026 • un laptop, un serveur CPU, aucun cloud

Vous appuyez sur Entrée.

Que se passe-t-il, vraiment ?

Vous utilisez un LLM tous les jours. Aujourd'hui, on ouvre la boîte.

À main levée :

Qui code avec un assistant IA ? · Qui sait ce qu'est un token ? · Qui a déjà fait tourner un modèle chez soi ?

Le programme : comprendre, agir, étendre

ACTE 1

Comprendre

Un LLM, vraiment : tokens, contexte, sampling, quantification. Et on en télécharge un en direct.

ACTE 2

Agir

Le développement agentique : la boucle, les outils. Un agent corrige un vrai bug, branché sur un modèle local.

ACTE 3

Étendre

Skills, MCP, paysage des harnais. On écrit une skill en direct et on l'invoque.

... et à chaque acte, une démonstration en direct. À la fin : tout ça tourne chez vous.

ACTE 1 • COMPRENDRE

Comprendre un LLM, vraiment



Comprendre

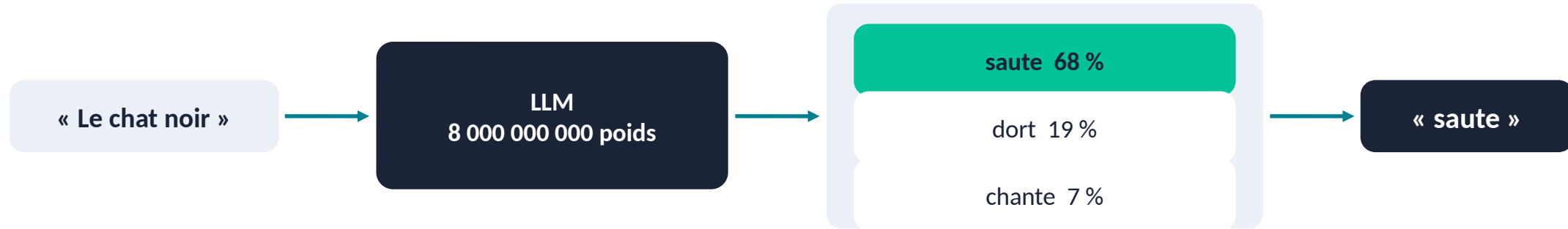


Agir



Étendre

Un LLM prédit le token suivant. C'est tout.



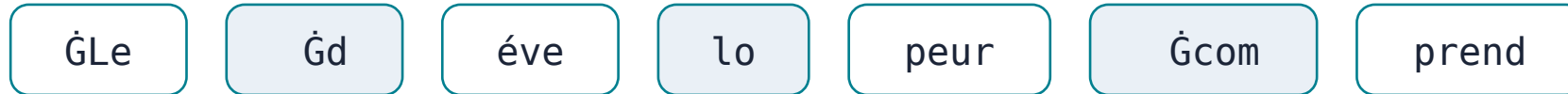
Le token choisi est ajouté à l'entrée, et le modèle recommence. Token après token. Il ne « sait » rien d'autre que prédire la suite la plus probable.

Pourquoi c'est important

- Pas de base de connaissances : des probabilités apprises.
- Une réponse = des milliers d'appels du même moteur.
- Toute la suite de la session repose sur cette phrase.

Le token : la monnaie du modèle

« Le développeur comprend » vu par le modèle :



Les ordres de grandeur

- ≈ 4 caractères = 1 token en anglais.
- Le français coûte 20-30 % de tokens de plus.
- « développeur » $\rightarrow$ « d » + « éve » + « lopeur » : le modèle ne voit pas des mots, des morceaux.
- Facturation, fenêtre de contexte, lenteur : tout se compte en tokens.

Chez vous aussi

Le petit modèle du live n'a pas de mot « français » : il a des morceaux de français, appris comme les autres. La langue n'est pas une option de compilation.

« 8B paramètres », ça veut dire quoi ?

8 000 000 000

nombre appris pendant l'entraînement — c'est tout ce qu'est le « cerveau »

- Chaque token qui traverse le modèle est multiplié contre ces nombres, couche après couche.
- Le code du modèle est ouvert ; ce qui compte, ce sont les poids — de vrais fichiers, téléchargeables.

Conséquence directe

Un poids est un nombre, pas un circuit : il peut être calculé par un CPU. Avec moins de précision (quantification, juste après), un 8 milliards de poids tient sur un laptop.

8B = un modèle de « poche ». Les modèles frontières en font 10 à 100 fois plus — là, il faut une salle de serveurs.

La fenêtre de contexte : sa mémoire de travail

Prompt système

AGENTS.md / règles du projet

Fichiers versés

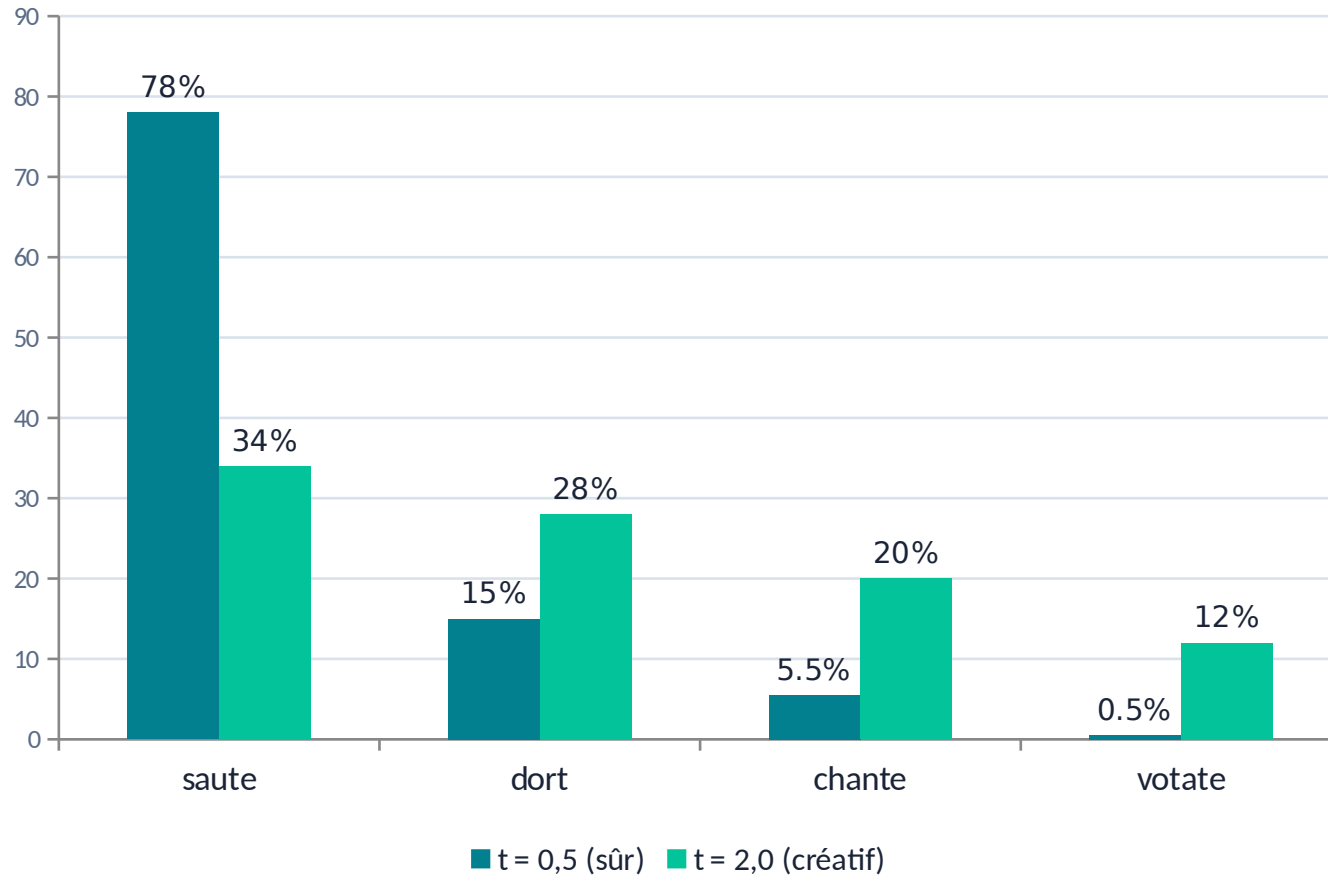
Historique de la conversation

Votre question

Ce que ça implique

- Le modèle ne « retient » rien entre deux sessions : tout doit tenir dans la fenêtre.
- Plus la conversation grossit, moins il reste de place pour le code.
- Ordres de grandeur : 8k (2023) → 128k → 262k chez nous.
- **Le contexte est une ressource rare → acte 2.**

La température : un dé plus ou moins pipé



Lire le graphique

- t basse : le modèle écrase sa confiance sur un seul token. Répétable, ennuyeux, prévisible.
- t haute : la distribution s'aplatit — « votate » devient possible.
- $t = 0$: purement déterministe (le même token à chaque fois).

Code à produire : température basse. Poésie : haute.

L'hallucination n'est pas un bug

Ce que le modèle fait

Prolonger le plausible. « API la plus probable après cette phrase » ≠ « API qui existe ».

Ce qu'il n'a pas

Un mode « je vérifie ». Pas de base de faits à consulter, pas de signal de certitude fiable.

D'où la doctrine : on ne demande pas au modèle d'être sûr, on lui donne de quoi vérifier — tests, compilateur, outils. C'est tout l'acte 2.

Comment on fabrique un LLM

1 · Pré-training

Des billions de pages du web. Objectif unique : prédire le token suivant. Naît un modèle qui « parle », mais qui finit tout.

2 · SFT

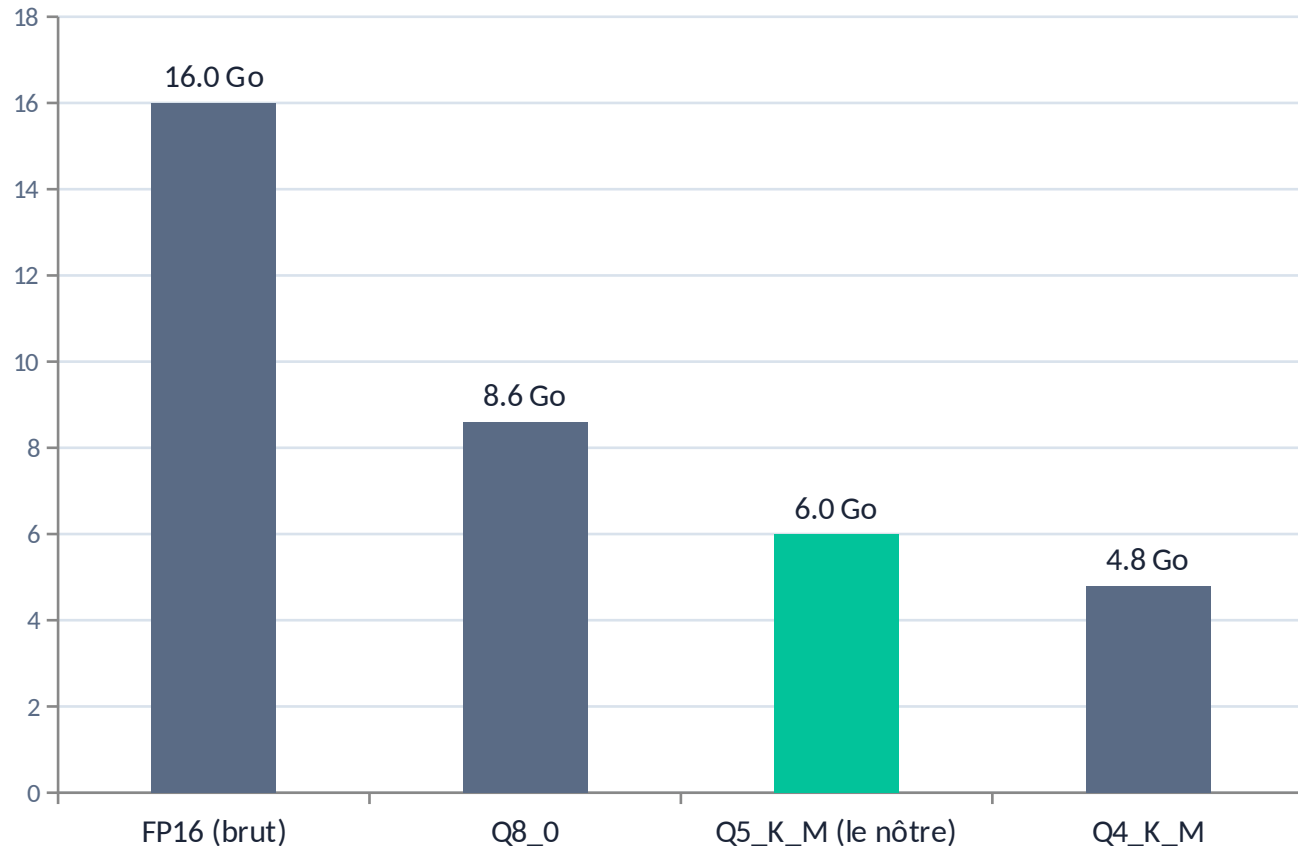
Fine-tuning sur conversations exemplaires : on lui apprend le format question → réponse, les outils, le ton.

3 · Alignement (RLHF)

Des humains comparent des réponses ; le modèle apprend à préférer ce qui est utile et honnête.

Un modèle « instruct » que vous téléchargez a passé les trois étapes. Le pré-training coûte des millions ; les deux autres, des semaines. Vous, vous prenez le résultat.

La quantification : 16 Go $\rightarrow$ 6 Go, presque la même tête



Le triangle

Taille • Vitesse • Qualité : en réduire un paye les autres.

- Q4-Q5 : la perte de qualité est faible pour un gain énorme en mémoire et vitesse CPU.

- GGUF = le format de llama.cpp, avec les variantes de quantification alignées sur la RAM dispo.

- **Le Q5_K_M de la slide : 6,0 Go mesurés sur le disque du serveur.**

LIVE

Hugging Face & llama.cpp, en direct

1 La page du modèle

huggingface.co/unsloth/LFM2.5-8B-A1B-GGUF : les fichiers GGUF par quantification, comme vu slide précédente.

2 hf download, en direct

On télécharge un petit modèle (~400 Mo) devant vous. Le temps de ma phrase.

3 llama-cli répond

Sur le CPU du laptop. Pas de clé API, pas de facture, pas de réseau — sauf pour le téléchargement.

```
$ hf download unsloth/LFM2.5-8B-A1B-GGUF \
  --include "...UD-Q5_K_M.gguf*" \
  --local-dir /models/LFM2.5-8B-A1B
```



Comprendre



Agir



Étendre

ACTE 2 • AGIR

Développement agentique : les harnais



Comprendre

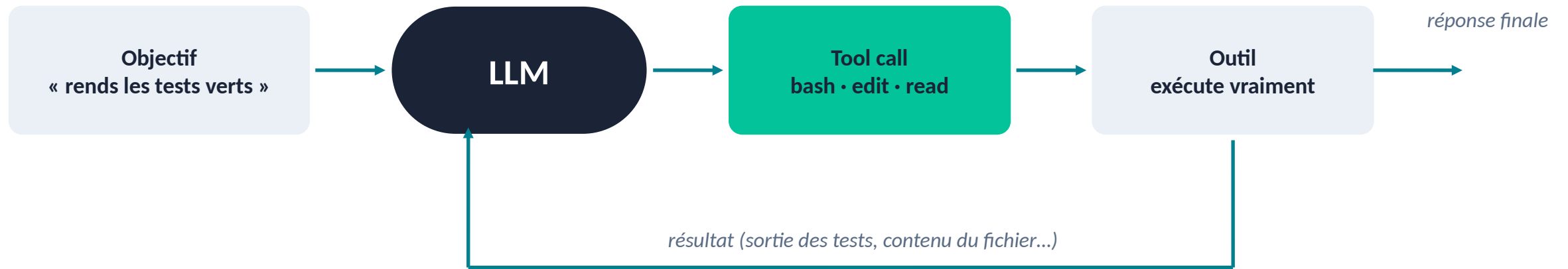


Agir



Étendre

Du chat à l'agent : la boucle



L'innovation n'est pas dans le LLM — c'est le même moteur qu'à l'acte 1. **Tout est dans la boucle : un modèle + des outils + un critère de réussite mesurable.**

Tool calling : appuyer sur des boutons

Le harnais déclare ses outils (schéma JSON) ; le modèle ne renvoie pas du texte libre, mais un appel structuré :

```
{  
  "name": "bash",  
  "input": {  
    "command": "python3 -m unittest"  
  }  
}
```

Le modèle n'exécute rien.

C'est le harnais qui lance la commande, récupère la sortie et la réinjecte dans le contexte du tour suivant.

Attention : tous les modèles n'y arrivent pas aussi bien. Un petit modèle local peut rater ses appels d'outils — c'est un critère de choix, pas un détail. (On vérifie ça au live suivant.)

Le contexte est la ressource rare

Rappel de l'acte 1 : chaque tour repaie toute la fenêtre. Un agent, c'est une conversation qui grossit vite :



fenêtre de contexte (262 144 tokens ici)

Ce que font les bons harnais (et les bons drivers)

- Lisent peu : grep avant lecture complète, extraient plutôt que verser.
- Compactent : résument les tours anciens pour libérer de la fenêtre.
- Externalisent : notes, fichiers, skills — ce qui doit survivre sort de la conversation.
- **Un AGENTS.md court et précis bat un pavé : chaque ligne est payée à chaque tour.**

Pourquoi ça ralentit — et à quelle vitesse ça va

Le KV cache, en une image

Pour ne pas recalculer tout le passé à chaque token, le modèle met en mémoire ses calculs intermédiaires (les clés et valeurs d'attention).

- Plus la conversation est longue, plus le cache est gros : il mange la mémoire vive avant la fenêtre théorique.
- Le premier token d'une longue requête est lent (prefill) ; les suivants sont rapides (décode).
- Un agent qui lit dix fichiers « paye » ces dix fichiers à chaque tour suivant.

≈ 35 tok/s

génération — llama-server sur CPU, mesuré

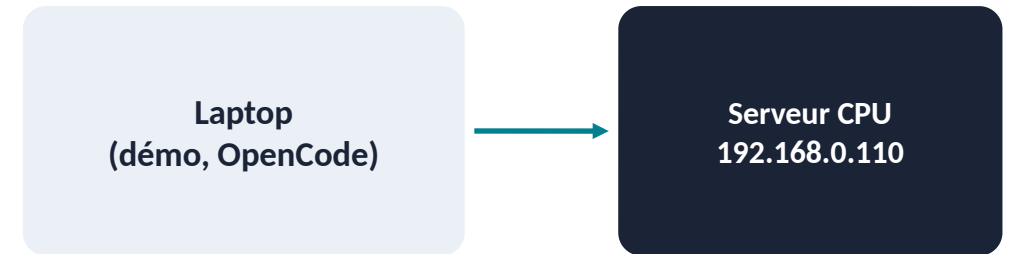
262 144

tokens de fenêtre sur notre serveur local — de quoi tenir
une vraie session agentique

Le branchement : dix lignes de configuration

```
"providers": {  
  "llama.cpp": {  
    "package": ".../openai-compatible",  
    "settings": {  
      "baseUrl":  
        "http://192.168.0.110:8080/v1"  
    }  
  }  
}
```

opencode.jsonc — extrait



API compatible OpenAI, sur le LAN

Le cloud, remplacé par :

une adresse IP locale. N'importe quel harnais qui parle l'API OpenAI se branche ici — mêmes outils, mêmes usages, données qui ne sortent pas.

LIVE

OpenCode + modèle local : 3 tests rouges

1 Le constat

python3 -m unittest dans demo/ : 9 tests, 3 échecs. Un bug de facturation. Je ne dis pas où.

2 L'agent travaille

« Les tests sont rouges, rends-les verts. » — lire, diagnostiquer, éditer, révéifier. Commenter les tool calls en direct.

3 git diff

Une valeur a changé : le palier de remise à 9 % au lieu de 10 %... d'après le code. La docstring, elle, disait la vérité.

L'agent qui fait ça tourne sur le serveur local, à 35 tokens/s. La session qui a PRÉPARÉ cette présentation aussi.



Comprendre



Agir



Étendre

ACTE 3 • ÉTENDRE

Skills, MCP, paysage



Comprendre



Agir



Étendre

Une skill = un fichier markdown

```
---  
name: explain-file  
description: Explique un fichier  
             de code (rôle, structure,  
             subtilités, limites)  
---  
1. Lis le fichier en entier.  
2. Rends : Rôle, Structure,  
           Subtilités, Limites.  
3. Moins de 20 lignes.
```

`.opencode/skills/explain-file/SKILL.md`

Comment ça marche

- Le harnais liste name + description à l'agent — c'est tout ce qui coûte du contexte au repos.
- Quand la situation colle, l'agent charge le corps du fichier et l'applique.
- Progressive disclosure : la connaissance est versée à la demande, pas en permanence.
- **Rien à re-entraîner, rien à déployer. Un fichier. Git-able, partageable, relisible dans dix ans.**

LIVE

Écrire une skill en direct

1 mkdir + SKILL.md

Un dossier, un fichier, le frontmatter (name + description). On le tape devant vous.

2 Quatre consignes

Rôle · Structure · Subtilités · Limites. La skill entière fait quinze lignes.

3 L'invocation

« explique demo/pricing.py » — l'agent voit la nouvelle skill, la charge, l'applique.

```
> explique demo/pricing.py  
skill: explain-file chargée ✓
```



Comprendre

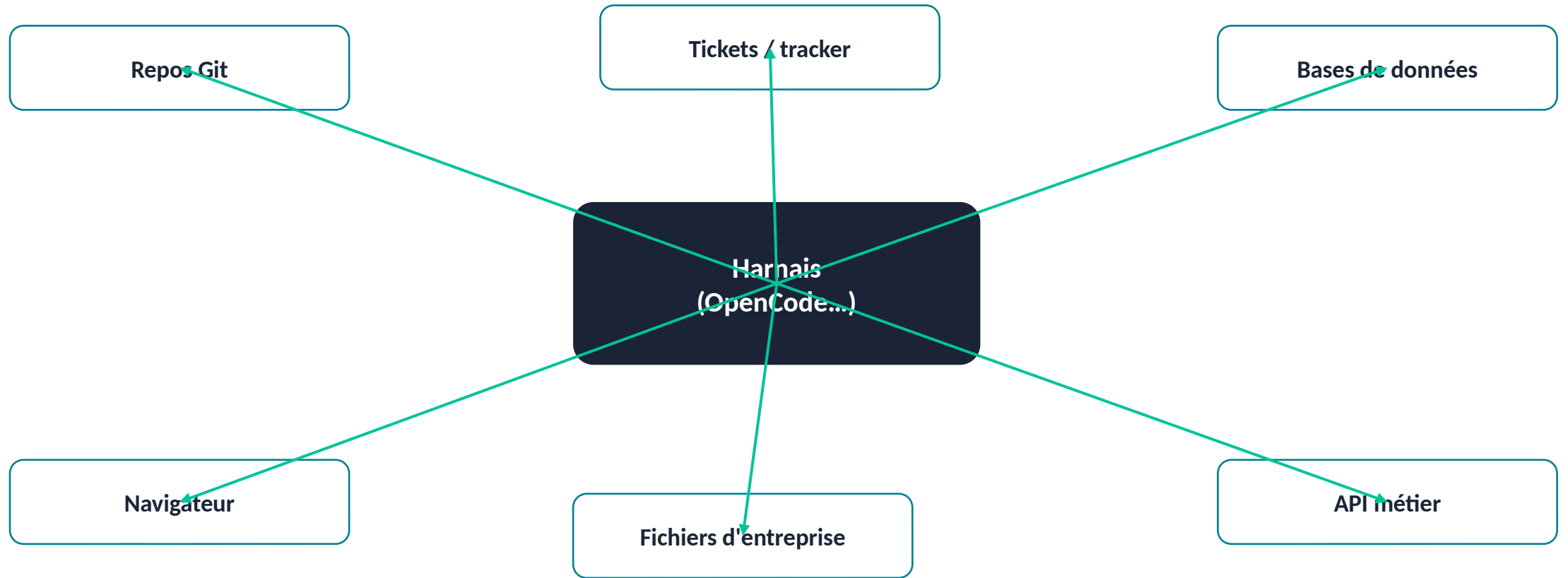


Agir



Étendre

MCP : le port USB des outils



Un serveur MCP expose des outils ; tous les harnais qui parlent MCP les voient. Standard ouvert, indépendant du vendeur.

Le paysage des harnais (octobre 2026)

Outil	Interface	Code	Modèle local
OpenCode	terminal + IDE	open source	oui (comme ici)
Claude Code	terminal	propriétaire	oui (serveur compatible)
Codex CLI	terminal	open source	oui (provider custom)
Gemini CLI	terminal	open source	partiel
Cursor	IDE	propriétaire	non (cloud d'abord)
Cline	extension VS Code	open source	oui

Terminal ↔ IDE, vertical ↔ éditeur étendu : la vraie ligne de partage est la dernière colonne.

Frameworks & standards — sans obligation

Frameworks d'agents

LangChain / LangGraph • CrewAI • AutoGen • PydanticAI

Pour CONSTRUIRE votre propre agent dans du code : workflows multi-agents, routage, états durables. Pertinent quand vous vendez de l'agentique — pas pour coder au quotidien.

Standards ouverts

MCP • AGENTS.md • format Agent Skills

Des conventions, pas des bibliothèques : les outils se conforment, vous changez de harnais sans rien perdre. Ce sont eux qui durent.

Pour démarrer, rien de tout ça n'est requis : **un harnais + un modèle local. Le reste est optionnel.**

Trois modèles mentaux à emporter

1

Machine à prédire

Le token suivant, encore et encore.
L'hallucination est structurelle : on
utilise la vérification, on ne la demande
pas au modèle.

2

La boucle

Un agent = LLM + outils + critère
mesurable. Le LLM ne change rien ; c'est
le montage qui développe.

3

Contexte rare

Chaque token dans la fenêtre est payé à
chaque tour. Bien coder avec un agent,
c'est gérer ce budget.

Bilan confidentiel — l'inventaire de la journée :

RIEN.

Vos prompts, le code de la démo, les tests, la skill, les correctifs de l'agent :
tout est resté sur les machines de cette salle.

Chez vous ce soir — trois commandes

```
$ git clone <le repo de cette présentation>
$ docker compose build
$ docker compose run --rm llama bash
# hf download <modèle> --local-dir /models/<modèle>
# llama-cli -m /models/<modèle>/<fichier>.gguf
```

QR → repo public
github.com/<vous>/llama

(à générer avant le jour J)

Le repo contient tout : conteneur llama.cpp, la démo avec son bug re-verdissable, la skill explain-file, le plan et les scripts des lives.

Merci.

Place aux questions — et si le serveur tient, à une démo improvisée.

repo : github.com/<vous>/llama · licence MIT · tout est rejouable